

Basics of COAP

- CoAP–Constrained Application Protocol.
- CoAP is an IoT Protocol.
- CoAP is designed to allow simple devices to join IoT through the low bandwidth–restricted network.
- CoAP is designed for M2M and IoT applications.
- CoAP is an Application Layer Protocol that follows the request–response model.
- CoAP runs over UDP Protocol.
- CoAP uses fewer resources than HTTP.
- COAP client can use GET, PUT, and DELETE commands during.

COAP Layers

CoAP is divided into two layers.

- Upper Layer (Application Layer) is designed for the communication method based on the Request–Response model.
- Lower Layer (UDP) is designed to deal with UDP and asynchronous messages.

Message Types COAP

CoAP Supports Four different message types:

1. Confirmable Message (Reliable Message) (Con):

- A Confirmable message requires a response, either a Positive Acknowledgement or a Negative Acknowledgement.
- If acknowledgement is not received then retransmissions are made until all attempts are exhausted.
- The Acknowledgement message contains the same ID as the confirmation message.
- If the server has trouble managing the incoming request, it message (RST) instead of an acknowledge message (ACK).

2. Non-confirmable Message {Unreliable Message} (Non):

- A Non-confirmable message doesn't require a response in the form of Acknowledgement.
- These messages do not contain critical information, like a request for a sensor measurement made in periodic basis.
- Even if one value is missed, there is not too much impact.
- Even if these messages are unreliable, they have a unique ID.

3. Acknowledgement (Ack):

- It is sent to acknowledge a confirmable message (Con).
- Confirms receipt of a CON message, carrying response data or error codes (e.g., 2.05 Content, 4.04 Not Found).
- Matches the CON's message ID to ensure correct pairing.
- Example: A server responds with an ACK containing 500 lux to a CON GET /sensors/light.

4. Reset (RST):

- It represents negative Acknowledgment.
- It generally indicates some kind of failure, like unable to process received data.
- Sent when a message is received but cannot be processed (e.g., unknown resource, malformed request).
- Indicates rejection or error without further action.
- Example: A client receives an RST if it sends a GET to a non-existent resource /sensors/invalid.

Message Structure:

- o Header: 4 bytes (Version, Type, Token Length, Code, Message ID).
- o Token: Correlates requests and responses (0– 8 bytes).
- o Options: Metadata (e.g., Uri-Path: sensors/temp, Content-Format: text/plain).
- o Payload: Data (e.g., 25.5° C).

Process:

1. Request: Client sends a request with a method (e.g., GET /sensors/temp), message type (CON or NON), and token.
2. Response: Server replies with a response code (e.g., 2.05 Content for success, 4.04 Not Found for errors) and payload.
3. Methods:
 - GET: Retrieves resource state (e.g., current temperature).
 - POST: Creates or updates a resource (e.g., sets a new threshold).
 - PUT: Updates a resource (e.g., changes device mode).

- DELETE: Removes a resource (e.g., deletes a log entry).
4. Observe: Clients send a GET with an Observe option (value 0) to subscribe to resource updates; the server notifies clients when the resource changes (e.g., temperature rises above 30°C).
 5. Block-Wise Transfer: Large data (e.g., firmware updates) is split into blocks, with options like Block1 (request) and Block2 (response).
- Example:

A CoAP client sends a CON GET /sensors/humidity with token 0xAB12.

The server responds with an ACK, code 2.05 Content, and payload 60% RH, using the same token.

If the client observes /sensors/humidity, the server sends NON messages whenever humidity changes significantly.

Applications

1. Smart Homes: Control devices (e.g., lights) in Thread or 6LoWPAN networks.
2. Smart Cities: Monitor parking spaces or air quality with low-power sensors.
3. Industrial IoT: Real monitoring of machine parameters over constrained networks.
4. Wearables: Low-power data exchange for health sensors (e.g., pulse monitors).
5. Environmental Monitoring: Collect data from remote sensors (e.g., river water levels).
6. Building Automation: Manage HVAC or lighting with multicast commands.

XMPP (Extensible Messaging and Presence Protocol)

XMPP is an open, XML-based protocol for real-time messaging, presence, and request response communication. Initially developed for instant messaging, it is adapted for IoT through extensions (XEPs) for device communication and control.

Basics of XMPP

- XMPP-eXtensible Messaging and Presence Protocol
- It is a protocol (standard) that is used to build chat systems.
- It uses XML to exchange data between client and server.
- eXtensible: The protocol can be (has been) extended with new features.
- Messaging: Send one-to-one and group messages.
- Presence: See your contact's online status.

XMPP Stanzas

Stanza: It is the basic unit of communication in XMPP.

There are three types of stanzas in XMPP

1. Message: used when you want to send messages
2. Presence: used to send online status information and to control subscription status between contacts.
3. IQ (Info/Query): used to [get] some information from the server or to [set] or apply some settings.

XMPP Features

1. Real-Time Communication: Delivers messages and presence updates instantly (e.g., device status changes).

2. Extensibility: XML stanzas support custom extensions via XMPP Extension Protocols (XEPs) for IoT (e.g., XEP-0323 for sensor data, XEP-0325 for control).
3. Decentralized Architecture: Federated servers (no central authority) enhance reliability and scalability.
4. Security: Supports TLS for encryption, SASL for authentication, and end-to-end encryption (e.g., OMEMO).
5. Peer-to-Peer Sessions: XMPP supports machine-to-machine or peer-to-peer across diverse set of networks.
6. Presence Management: Tracks device availability (e.g., online, offline, busy).
7. Bidirectional Streaming: Persistent TCP connections allow continuous client-server communication.
8. Multi-Device Support: A single JID (e.g., device@domain.com) can represent multiple devices or resources.
9. High Overhead: XML-based messages are verbose, less suitable for ultra-constrained devices.

Components and Communication.

Multi-User chat: XMPP supports group and conference chat.

1. Encryption: Point-to-point encryption is done by TLS {Transport Layer Security} and OTR {Off The Record messaging} is an extension of XMPP that enables encryption of messages and data.
2. Connection to other protocols: using XMPP Gateways other protocols like SMS and SMTP can also be connected with XMPP servers for different services.

Components:

- o Client: Device or application identified by a JID (Jabber ID, e.g., sensor1@domain.com/resource).
- o Server: Routes stanzas and manages presence (e.g., Openfire, ejabberd, Prosody).
- o Stanzas: XML data units for communication:

Message: Carries data (e.g., <message><body>{"temp": 25}</body></message>).

Presence: Indicates status (e.g.,

<presence><status>online</status></presence>).

IQ (Info/Query): Request-response for queries or commands (e.g., <iq type="get"> to retrieve device config).

- o Domain: Logical grouping of clients (e.g., domain.com).

• Communication:

1. Connection: Client establishes a TCP connection to a server, authenticates via SASL (e.g., with a password or certificate).
2. Stanza Exchange: Clients send stanzas to other JIDs or servers:
 - Message: Sends data (e.g., sensor readings to an app).
 - Presence: Broadcasts status (e.g., device online after boot).
 - IQ: Queries or sets data (e.g., GET for current sensor value, SET for actuator control).
3. Routing: Servers forward stanzas based on JIDs, supporting federation across domains.
4. Extensions: IoT-specific XEPs enable sensor data (XEP-0323), control (XEP-0325), or provisioning (XEP-0324).

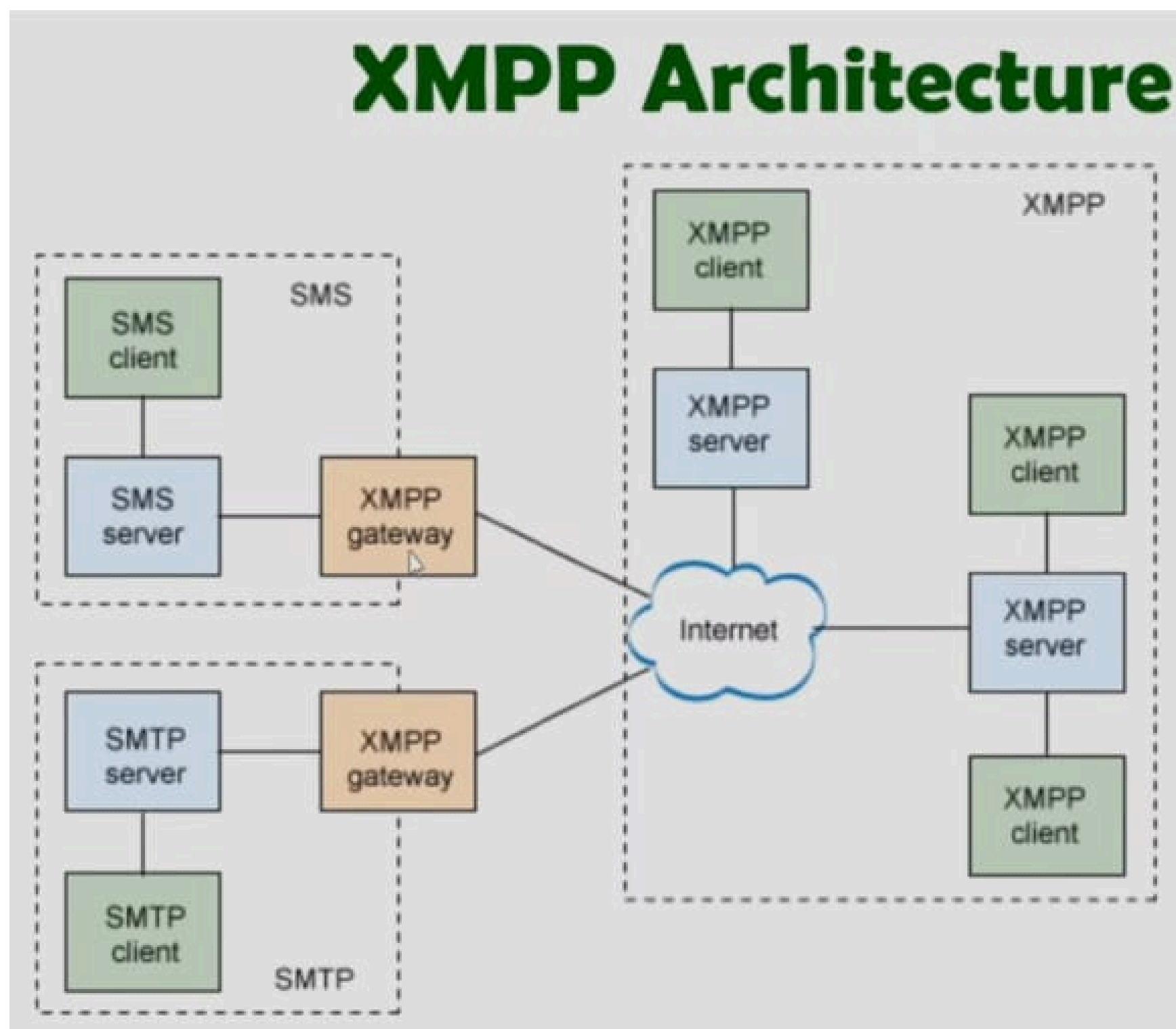
Example:

- o A smart thermostat (thermostat@home.com) sends a <message> stanza with {"temp": 22.5} to a mobile app (user@home.com).
- o The app responds with an IQ stanza to set the thermostat to 24° C.
- o The server broadcasts a <presence> stanza when the thermostat goes offline.

Applications

1. Smart Homes: Real-time control of lights, locks, or thermostats with presence updates.
2. Industrial IoT: Machine status notifications to operators via XMPP servers.

3. Healthcare: Patient monitoring with alerts for abnormal vital signs.
4. Chat-Based IoT: Integrates devices with messaging platforms (e.g., control a lamp via a Chatbot).
5. Smart Grids: Coordinates energy devices with real-time messaging and presence.
6. Collaborative IoT: Multi-user device control in shared environments (e.g., office automation).



The image provided is a diagram of the XMPP Architecture (Extensible Messaging and Presence Protocol), showing how XMPP integrates with other communication systems like SMS and SMTP (email).

1. XMPP Clients and Servers (Right side)

- XMPP Clients: These are applications or users connected to the XMPP network (e.g., messaging apps like Jabber, Pidgin, etc.).
- XMPP Servers: These manage communication between XMPP clients. They handle message delivery, presence info, and routing.
- Communication: Clients connect to servers. Servers communicate with each other via the Internet to allow messages to pass across domains.

2. Gateways (Center)

- XMPP Gateways act as bridges between XMPP and other communication systems (e.g., SMS, Email/SMTP).
- They translate the message formats and protocols so users in the XMPP system can communicate with users on other systems (like sending an XMPP message as an SMS or email).

3. SMS System (Left top)

- SMS Client: A user or device sending/receiving text messages.
- SMS Server: Handles SMS message routing.
- XMPP Gateway: Connects the SMS server to the XMPP network, allowing SMS users to interact with XMPP users.

4. SMTP System (Left bottom)

- SMTP Client: An email-sending application (e.g., a mail client).
- SMTP Server: Sends and receives emails using SMTP protocol.

- XMPP Gateway: Enables communication between email systems and the XMPP network (e.g., XMPP user can send a message that is received as an email).

Internet Connectivity

All systems (XMPP servers, SMS/XMPP gateways, SMTP/XMPP gateways) are connected via the Internet, ensuring interoperability and communication across platforms.